Italian Restaurant Sales Analytics

02 - Promotions Data Cleaning and Preparation

Objective

This notebook cleans and validates the **Promotions** worksheet from the Italian Restaurant Sales Analytics dataset. It verifies data quality, checks for missing values and duplicates, validates data types, and exports a cleaned dataset for further analysis in Python, SQL, and Power BI.

Step 1 - Import Libraries

The first step is to import the Python libraries required for data manipulation and analysis.

In [1]:
```
# Import pandas for data manipulation and analysis
import pandas as pd
```

Step 2 - Load the Excel Workbook

The next step is to define the location of the Excel file and load the workbook into Python.

In [2]:
```
# Define the relative path to the Excel workbook
file_path = "../data/raw/restaurant-sales-analytics-data.xlsx"
```

Step 3 - Read the Promotions Worksheet

Load the Promotions worksheet into a pandas DataFrame.

In [3]:
```
# Load the Promotions worksheet into a pandas DataFrame
promotions = pd.read_excel(file_path, sheet_name="Promotions")
```

Step 4 - Preview the Dataset

Display the first five rows of the dataset.

In [4]:
```
# Display the first five rows
promotions.head()
```

Out[4]:

	Date	Promotion	Uses	Discount Amount
0	2025-03-01	15% DESC. EMPLEADOS	23	67.92
1	2025-03-01	100% PROPIETARIOS	34	537.74
2	2025-03-01	3x2 SIEMPRE	258	2622.20
3	2025-03-01	CORTESIA AL PRODUCTO	35	191.30
4	2025-03-01	DESC MODO TASTY 50%	61	656.02

Step 5 - Inspect the Dataset Structure

Display the structure of the dataset, including data types, number of non-null values, and memory usage.

In [5]:
```
# Display dataset information
promotions.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 147 entries, 0 to 146
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Date             147 non-null    datetime64[ns]
 1   Promotion        147 non-null    object
 2   Uses             147 non-null    int64
 3   Discount Amount  147 non-null    float64
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 4.7+ KB
```

Step 6 - Check Dataset Dimensions

Display the number of rows and columns in the dataset.

In [6]:
```
# Display dataset dimensions
promotions.shape
```

Out[6]: (147, 4)

Step 7 - Check Missing Values

Identify whether the dataset contains any missing values in each column.

In [7]:
```
# Check for missing values
promotions.isnull().sum()
```

Out[7]:
```
Date              0
Promotion         0
Uses              0
Discount Amount   0
dtype: int64
```

Step 8 - Check for Duplicate Rows

Identify whether the dataset contains duplicate records that may affect the analysis.

In [8]:
```
# Check for duplicate rows
promotions.duplicated().sum()
```

Out[8]:
```
np.int64(0)
```

Step 9 - Display Column Names

Display all column names in the dataset.

In [9]:
```
# Display column names
promotions.columns
```

Out[9]:
```
Index(['Date', 'Promotion', 'Uses', 'Discount Amount'], dtype='object')
```

Step 10 - Display Summary Statistics

Display descriptive statistics for the numerical columns to better understand the distribution of the data.

In [10]:
```
# Display summary statistics
promotions.describe()
```

Out[10]:

	Date	Uses	Discount Amount
count	147	147.000000	147.000000
mean	2025-08-31 06:02:26.938775552	56.428571	613.534218
min	2025-03-01 00:00:00	1.000000	0.880000
25%	2025-05-16 12:00:00	5.000000	35.905000
50%	2025-09-01 00:00:00	15.000000	82.570000
75%	2025-12-01 00:00:00	38.000000	596.880000
max	2026-03-01 00:00:00	415.000000	4945.450000
std	NaN	91.520639	1097.900464

Step 11 - Check Unique Promotions

Display the unique values in the Promotion column to verify that all expected categories are present.

In [17]:
```
# Display unique promotions
promotions["Promotion"].unique()
```

Out[17]:
```
array(['15% DESC. EMPLEADOS', '100% PROPIETARIOS', '3x2 SIEMPRE',
       'CORTESIA AL PRODUCTO', 'DESC MODO TASTY 50%',
       '15 % DESC. WHISKYS', '30% PROPIETARIOS', '4x3 STELLA',
       '%DESC. AJUSTE APPS', 'V&E&L FEBRERO 2026', '10% GRANDE TABLE',
       '3x2 OUT 4-7PM', '25% WISHKYS PREMIUM', '15% DESC. BOCATTO',
       'DESCUENTO CERTIFICADO DE CONSUMO', '10% DESCUENTO VIVE MAS',
       '10% DESCUENTO BARRA', '10% DESC. DINERS', 'DESCUENTO VINOS 20%',
       '30 % DESC. B.I.', '50% MODO TASTY'], dtype=object)
```

Step 12 - Count Unique Promotions

Count the number of unique Promotions in the dataset.

In [16]:
```
# Count unique promotions
promotions["Promotion"].nunique()
```

Out[16]:
```
21
```

Step 13 - Count Unique Dates

Count the number of unique dates in the dataset.

In [15]:
```
# Count unique dates
promotions["Date"].nunique()
```

Out[15]:
```
13
```

Step 14 - Count Unique Discount Amount Values

Count the number of unique discount amount values in the dataset.

In [ ]:
```
# Count unique discount amount values
promotions["Discount Amount"].nunique()
```

Out[ ]:
```
146
```

Step 15 - Display Data Types

Display the data type of each column individually.

In [19]:
```
# Display data types
promotions.dtypes
```

Out[19]:
```
Date               datetime64[ns]
Promotion                  object
Uses                        int64
Discount Amount           float64
dtype: object
```

Step 16 - Check for Negative Discount Amounts

Verify whether the dataset contains any negative discount amount values.

In [20]:
```
# Check for negative discount amount values
(promotions["Discount Amount"] < 0).sum()
```

Out[20]:
```
np.int64(0)
```

Step 17 - Check for Zero Discount Amount Values

Verify whether the dataset contains Discount Amount values equal to zero.

In [21]:
```
# Check for zero discount amount values
(promotions["Discount Amount"] == 0).sum()
```

Out[21]:
```
np.int64(0)
```

Step 18 - Verify Date Range

Display the earliest and latest dates in the dataset.

In [22]:
```
# Display minimum and maximum dates
promotions["Date"].min(), promotions["Date"].max()
```

Out[22]:
```
(Timestamp('2025-03-01 00:00:00'), Timestamp('2026-03-01 00:00:00'))
```

Step 19 - Sort the Dataset by Date

Sort the dataset in chronological order.

In [23]:
```
# Sort the dataset by date
promotions = promotions.sort_values(by="Date")
```

Step 20 - Reset the Index

Reset the DataFrame index after sorting.

In [24]:
```
# Reset the DataFrame index
promotions.reset_index(drop=True, inplace=True)
```

Step 21 - Preview the Clean Dataset

Display the first rows of the cleaned dataset.

In [25]:
```
# Preview cleaned dataset
promotions.head()
```

Out[25]:

	Date	Promotion	Uses	Discount Amount
0	2025-03-01	15% DESC. EMPLEADOS	23	67.92
1	2025-03-01	15% DESC. BOCATTO	4	52.28
2	2025-03-01	25% WISHKYS PREMIUM	4	59.71
3	2025-03-01	3x2 OUT 4-7PM	1	9.90
4	2025-03-01	10% GRANDE TABLE	185	1857.05

Step 22 - Export the Clean Dataset

Export the cleaned dataset as a new Excel file for future analysis.

In [26]:
```
# Export cleaned Promotions dataset
promotions.to_excel(
    "../data/cleaned/promotions_cleaned.xlsx",
    index=False
)
```

Step 23 - Cleaning Summary

Display a summary of the data quality checks performed during the cleaning process.

In [29]:
```
print("========== CLEANING SUMMARY ==========")

print(f"Rows: {promotions.shape[0]}")
print(f"Columns: {promotions.shape[1]}")
print(f"Missing values: {promotions.isnull().sum().sum()}")
print(f"Duplicate rows: {promotions.duplicated().sum()}")
print(f"Negative Discount Amounts: {(promotions['Discount Amount'] < 0).sum()}")
print(f"Zero discount amount: {(promotions['Discount Amount'] == 0).sum()}")
print(f"Date range: {promotions['Date'].min().date()} to {promotions['Date'].max().
```

```
print("Clean dataset exported successfully.")
print("=====================================")
```

```
========== CLEANING SUMMARY ==========
Rows: 147
Columns: 4
Missing values: 0
Duplicate rows: 0
Negative Discount Amounts: 0
Zero discount amount: 0
Date range: 2025-03-01 to 2026-03-01
Clean dataset exported successfully.
=====================================
```

Author

Guido Acosta

Data Analyst Portfolio Project

GitHub Portfolio – 2026